

Week 6 Resources: From Synthetic Markets to Real Data

Goal of Week 6

Week 5 was a controlled experiment: you put a known signal (ρ) into a synthetic market and confirmed your agent could find it. You now trust your pipeline — if there is learnable structure, your environment, training loop, and evaluation harness will surface it.

Week 6 points that trusted pipeline at real market data. Almost nothing changes in the code — the environment interface, PPO call, and evaluation harness are the ones you already have. What changes is everything around the code: the data is a single fixed history rather than a generator, the structure (if any) is unknown and weak, and the evaluation must respect the arrow of time.

By the end of this week you should be able to:

- Download, clean, and inspect real daily price data, and compute log returns.
- Explain why a train/test split on time series must be chronological, and what lookahead bias is.
- Build a Gymnasium environment that replays windows of historical returns.
- Interpret a train-vs-test performance gap as overfitting.
- State what a realistic, honest result looks like for an RL trading agent on real daily data — and why a null result is a finding, not a failure.

Recap: Key Ideas from Week 5

Before moving on, confirm you can answer these:

- Why is a pure random walk unbeatable, in one equation or two sentences?
- What is lag-1 autocorrelation and how do you measure it with NumPy?
- What was the optimal hand-coded rule on the momentum market, and did your agent find it?
- When is a signal not worth trading, in terms of edge and cost?

If any of these feel shaky, re-read the Week 5 resources before continuing.

What Changes with Real Data

Four differences between your synthetic markets and a real price series, roughly in order of how much they will affect your week:

1. **There is only one history.** In Week 5 you could generate unlimited fresh episodes. Real data is one fixed sequence of a few thousand days. Every episode your agent trains on is a window of the same history, so the agent can *memorise* it — which is why held-out test data becomes essential this week.

2. **The structure is unknown and weak.** You will measure the lag-1 autocorrelation of real daily returns in Task 1. Expect something very close to zero — far from Week 5’s $\rho = 0.5$. Whatever edge exists in real markets is small, unstable, and mostly invisible to a 5-return state window.
3. **There is drift.** Real equity indices have a small positive mean daily return (that is why long-term investors make money). Your synthetic returns had mean exactly zero. Drift means buy-and-hold is no longer a zero-expectation baseline — it becomes a genuinely hard one to beat.
4. **The rules change over time.** A quiet 2017 and a violent March 2020 are both in your dataset, and they behave very differently (statisticians say the series is *non-stationary*). An agent tuned to one regime can fail badly in another. You do not need to solve this — just recognise it when you see a train/test gap.

Log Returns

From a price series P_t , this week uses **log returns**:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right).$$

For small daily moves, r_t is nearly identical to the percentage change you have used so far (for a 1% move, they differ by about 0.005%), so nothing in your intuition changes. Log returns are standard because they add across time: the total log return over a window is just the sum of the daily ones, which makes cumulative calculations clean. In code it is one line: `np.diff(np.log(prices))`.

One practical note: use *adjusted* closing prices (`auto_adjust=True` in `yfinance`). These fold dividends and stock splits back into the price so that a 2-for-1 split does not appear as a fake -50% crash in your returns.

The Golden Rule: No Information from the Future

Everything about evaluating on time series follows from one rule:

At time t , you may only use information that existed at time t .

Violating it is called **lookahead bias**, and it is the single most common way backtests lie. Some violations are obvious (using tomorrow’s price in today’s state); most are subtle. The one you must avoid this week is in the split itself:

- **Chronological split (correct):** train on the first 80% of days, test on the last 20%. The test years are the agent’s “future” — it has never seen them, just as a deployed agent will not have seen its future.
- **Shuffled split (wrong):** randomly assign days to train and test. Now the training set contains days from *inside and after* the test period. The agent has effectively seen the test era’s behaviour — its volatility level, its trend — during training. Test performance becomes meaningless.

This is a real difference from ordinary supervised learning, where shuffling before a split is standard practice. On time series, shuffling is cheating.

Episodes from a Single History

Your `HistoricalTradingEnv` creates an episode by choosing a *random contiguous window* of 100 days from the data it was given. This has two purposes:

- **Variety for training.** PPO sees windows starting everywhere in the training years, in random order, rather than replaying one fixed episode. With $\sim 2\,000$ training days there are $\sim 1\,900$ possible windows.
- **Averaging for evaluation.** Evaluating over 50 random test windows gives a mean P&L curve, exactly as in Week 4, rather than a verdict from one lucky or unlucky stretch.

Be clear about what this trick does *not* do: the windows overlap heavily, so 50 evaluation episodes are not 50 independent samples, and no amount of resampling creates information beyond the one history you have. This is a fundamental limit of backtesting — one you should name in your notes rather than paper over.

Overfitting and the Train/Test Gap

This week’s central measurement is a comparison the previous weeks could not make:

$$P\&L \text{ on training data} \quad \text{vs.} \quad P\&L \text{ on test data.}$$

- **Train high, test near zero:** the classic overfitting signature. The agent memorised patterns of the training years — real patterns of that particular past, but ones that did not persist. On real financial data this outcome is *likely*, and observing it clearly is a successful week.
- **Train and test both near zero:** the agent found nothing worth memorising, and honestly says so on both sets. Also a valid result — consistent with the near-zero autocorrelation you measured in Task 1.
- **Test clearly positive across several runs:** be suspicious first (re-check the split, re-run with fresh seeds), impressed later. Extraordinary claims need more than one training run — this is Week 4’s “be suspicious of profit” lesson with sharper teeth.

Related vocabulary worth knowing: searching many configurations and reporting the best backtest is called *backtest overfitting* — with enough tries, something always looks profitable on a fixed history by luck alone. This is why Task 3 makes you run the experiment three times, and why the capstone will use multiple seeds as standard practice.

What a Good Result Looks Like This Week

Recalibrate expectations before you run anything. On real daily data, with a tiny state window, transaction costs, and ~ 8 years of training data, a PPO agent that consistently beats buy-and-hold out of sample would be remarkable — hedge funds employ large teams to chase edges this setup cannot see. The realistic outcomes are the first two bullets above, and both are genuinely useful findings.

What you are actually being graded on — by reality, not just your mentor — is the *process*: a clean chronological split, evaluation against baselines on unseen data, repeated runs before drawing

conclusions, and an honest write-up of what you found. That process is the transferable skill; it is exactly how any serious claim about a trading strategy (or any ML model on time series) gets checked.

Connecting Back to the Five Questions

As always, ground the new ideas in the core RL framework:

- **Agent:** The PPO policy network — unchanged since Week 3.
- **Environment:** `HistoricalTradingEnv` — real returns replayed in random windows, split chronologically into train and test worlds.
- **State:** $[r_{t-k+1}, \dots, r_t, \text{current_position}]$ — now real log returns instead of simulated ones.
- **Actions:** Discrete — 0 = Go Long, 1 = Hold, 2 = Go Short.
- **Reward:** $\text{position} \times r_t \times 100 - \text{cost} \times |\Delta \text{position}|$ — unchanged.

Notice how stable this framework has been: since Week 3, every step forward — history windows, costs, momentum, real data — has been a change to the *environment*, while the agent, the algorithm, and the five questions stayed the same.

Suggested External Resources (Optional)

You only need this document and the assignment to complete Week 6. If you want more context:

- yfinance documentation — the “download” section; you need nothing else from it.
- A short article on lookahead bias and backtest overfitting (search “backtest overfitting explained”). Blog-level is fine; skip the statistics.
- Stable-Baselines3 documentation — `model.save()` and `PPO.load()`, useful for the optional extension and essential for the capstone.

Self-Check Before Moving On

You are ready for the Weeks 7–8 capstone when you can:

- Download real price data, compute log returns, and report their mean, std, and lag-1 autocorrelation.
- Explain lookahead bias and why a chronological split prevents the worst of it.
- Train PPO on the training years and evaluate on held-out test years against the random and buy-and-hold baselines.
- Look at a train/test gap and say what it means — and defend a near-zero test result as an honest, correct finding.
- Explain why one training run proves nothing, and what you would report instead.